

SMART COMPLAINT MANAGEMENT SYSTEM

Empowering Citizens, Enabling Smarter Cities.

1. Cover Page

Field	Details
Project Title	Smart Complaint Management System
Tagline	Empowering Citizens, Enabling Smarter Cities.
Hackathon Theme	Smart City
Hackathon Name	Nexus Hackathon 2026
Team Name	Genesis5
Team Leader	Nutan Dhepe
Team Members	Dhanashri Nanekar, Tanuja Garje, Tanisha Mohite, Sanskruti Jadhav
Organization	Techspace Programming Institute
Location	Mumbai, Maharashtra
Date	2026
GitHub Repository	https://github.com/Nutan-14/nexus-hackathon-2026

[Space reserved for Hackathon Logo]

2. Table of Contents

- 1. Cover Page
- 2. Table of Contents
- 3. Executive Summary
- 4. Problem Statement
- 5. Proposed Solution
- 6. Objectives
- 7. Scope
- 8. System Architecture
- 9. Technology Stack
- 10. Features

11. Modules
12. Workflow
13. Database Design
14. UI Screens
15. Algorithms
16. Challenges Faced
17. Future Enhancements
18. Impact
19. Results
20. Conclusion
21. References
22. Appendix

3. Executive Summary

The **Smart Complaint Management System (SCMS)** is a web-based platform designed under the **Smart City** initiative to modernize the way citizens interact with municipal authorities. It replaces the traditional paperwork-driven complaint system with a fully digital, transparent, and trackable workflow.

- **Purpose:** Bridge the communication gap between citizens and civic bodies.
 - **Problem Solved:** Manual complaint registration, lack of tracking, delayed resolution, and poor transparency.
 - **Main Features:** Complaint registration with photos, live status tracking, admin dashboard, category-based filtering, and complaint history.
 - **Technology Used:** HTML5, CSS3, JavaScript (frontend); Python Flask (backend); MySQL (database); Git & GitHub (version control).
 - **Expected Benefits:** Faster complaint resolution, transparent governance, higher citizen participation, and data-driven civic decision making.
-

4. Problem Statement

Cities across India — particularly rapidly growing metros like Mumbai — face persistent challenges in civic complaint management. The **current system is largely manual and fragmented**.

4.1 Existing Problems

- **Manual Complaint Registration:** Citizens must physically visit municipal offices or make phone calls.
- **Delayed Response:** Paper-based routing causes days or weeks of delay.

- **Lack of Transparency:** Citizens cannot see who is handling their complaint or its current status.
- **Poor Communication:** No structured notification channel between the authority and the citizen.
- **Difficulty Tracking Complaints:** No unique ID or dashboard; complaints get lost.
- **Duplicate Complaints:** Same issue is reported multiple times, wasting resources.
- **No Analytics:** Authorities cannot identify hotspots or repeat issues.

4.2 Why This Problem Matters

Unresolved civic issues — potholes, water leakage, garbage overflow, broken street lights — directly affect public safety, hygiene, and quality of life. A slow complaint system erodes citizen trust in governance.

4.3 Who Is Affected

- **Citizens** — daily commuters, residents, students, senior citizens.
 - **Municipal Authorities** — overwhelmed by unstructured complaints.
 - **Government** — reduced credibility of Smart City programs.
-

5. Proposed Solution

The **Smart Complaint Management System** is a centralized, digital, two-portal platform:

5.1 How It Works

1. Citizens register on the portal and log in securely.
2. They submit a complaint with category, description, location, photo, and date.
3. The system stores it in a central MySQL database and assigns a unique Complaint ID.
4. Administrators view all complaints on an interactive dashboard.
5. Admins update status: **Open** → **In Progress** → **Resolved**.
6. Citizens track progress in real time from their dashboard.

5.2 Citizen Workflow

Register → Login → Submit Complaint → Track Status → View History.

5.3 Administrator Workflow

Login → Dashboard → View/Search/Filter Complaints → Update Status → Resolve.

5.4 Benefits

- Paperless, fast, and transparent.
- Complete audit trail for every complaint.

- Photo evidence improves accountability.

5.5 Unique Features

- Image-based complaint reporting.
- Category-based routing.
- Real-time status tracking with three-stage flow.
- Centralized admin control panel.

5.6 Innovation

Combines simplicity (lightweight Flask stack) with real municipal utility — deployable even in low-resource ULBs.

5.7 Real-World Application

Deployable in municipal corporations, panchayats, gated communities, university campuses, and industrial townships.

6. Objectives

1. **Improve Complaint Resolution Speed** by digitizing the entire lifecycle.
 2. **Reduce Paperwork** through fully electronic records.
 3. **Increase Transparency** with visible status tracking.
 4. **Promote Digital Governance** aligned with the Digital India mission.
 5. **Boost Citizen Engagement** via easy-to-use interfaces.
 6. **Support Smart City Development** by generating structured civic data.
 7. **Enable Easy Complaint Tracking** through unique complaint IDs.
 8. **Provide Centralized Complaint Management** for authorities.
-

7. Scope

7.1 Target Users

- Municipal Corporations (e.g., BMC, PMC)
- Government Organizations & Smart City SPVs
- Citizens and residents
- Urban Local Bodies (ULBs) and Panchayats
- Educational institutions for demonstration and academic use

7.2 Future Scalability

- Extendable to mobile apps (Android/iOS).
 - Cloud deployment (AWS/Azure/GCP).
 - Integration with GIS/Google Maps.
 - Multi-language and voice-based complaint entry.
 - AI-based auto-categorization and priority prediction.
-

8. System Architecture

8.1 High-Level Architecture (Mermaid)

```
flowchart LR
    U[Citizen Browser] -->|HTTPS| FE[Frontend<br/>HTML/CSS/JS]
    A[Admin Browser] -->|HTTPS| FE
    FE -->|REST Calls| BE[Flask Backend]
    BE --> DB[(MySQL Database)]
    BE --> FS[Image Storage]
```

Draw.io text version: Citizen/Admin Browser → Frontend (HTML/CSS/JS) → Flask Backend → MySQL Database + Image Storage

8.2 Low-Level Architecture (Mermaid)

```
flowchart TB
    subgraph Client
        C1[Login Page]
        C2[Complaint Form]
        C3[History Page]
        C4[Admin Dashboard]
    end
    subgraph Server[Flask Server]
        R1[Auth Controller]
        R2[Complaint Controller]
        R3[Admin Controller]
        R4[Validation Layer]
    end
    subgraph Data
        D1[(users)]
        D2[(complaints)]
        D3[(status_logs)]
        D4[/uploads/]
    end
    C1 --> R1 --> D1
    C2 --> R2 --> D2
    C2 --> R4
    C3 --> R2
    C4 --> R3 --> D2
    R3 --> D3
    R2 --> D4
```

8.3 Component Diagram

```
flowchart LR
    UI[Presentation Layer] --> API[Application Layer / Flask APIs]
    API --> BL[Business Logic]
    BL --> DAL[Data Access Layer]
    DAL --> DB[(MySQL)]
    BL --> FILE[File Storage]
```

8.4 Data Flow Diagram — DFD Level 0

```
flowchart LR
    Citizen((Citizen)) -->|Complaint Data| SCMS[Smart Complaint Management System]
    Admin((Admin)) -->|Status Updates| SCMS
    SCMS -->|Status/History| Citizen
    SCMS -->|Complaint List/Reports| Admin
    SCMS <--> DB[(MySQL DB)]
```

8.5 Workflow Diagram

```
flowchart TD
    A[Citizen Login] --> B[Fill Complaint Form]
    B --> C[Validate Input]
    C --> D[Save to DB]
    D --> E[Admin Dashboard]
    E --> F{Action?}
    F -->|In Progress| G[Update Status]
    F -->|Resolved| H[Close Complaint]
    G --> I[Citizen Tracks Status]
    H --> I
```

9. Technology Stack

Layer	Technology	Reason for Selection
Frontend	HTML5	Standard structural markup; lightweight and universal.
Frontend	CSS3	Responsive styling, animations, and modern UI.
Frontend	JavaScript	Client-side validation and interactivity.
Backend	Python Flask	Simple, lightweight, and fast micro-framework — ideal for hackathon MVPs.
Database	MySQL	Reliable relational database with strong integrity and easy schema design.
Version Control	Git & GitHub	Team collaboration, issue tracking, and code history.
IDE	Visual Studio Code	Rich extension ecosystem, integrated Git, and debugging.

10. Features

1. **Citizen Registration** – New users create accounts with name, email, phone, and password.
 2. **Secure Login** – Password-hashed authentication for both roles.
 3. **Complaint Registration** – Structured form for civic complaints.
 4. **Image Upload** – Attach photo evidence (JPG/PNG).
 5. **Location Selection** – Enter address / area to help authorities pinpoint issues.
 6. **Category Selection** – Garbage, Potholes, Water Leakage, Street Lights, Drainage, Illegal Parking, Noise Pollution.
 7. **Complaint Tracking** – Live view of Open / In Progress / Resolved status.
 8. **Dashboard** – Personalized dashboard for citizen and admin.
 9. **Complaint Search** – Search by ID, keyword, or category.
 10. **Status Update** – Admin-only three-stage state machine.
 11. **Complaint History** – Full log of prior complaints per citizen.
 12. **Admin Panel** – Comprehensive control center.
 13. **Reports** – Aggregate stats by category, status, and time.
 14. **Responsive Design** – Works on mobile, tablet, and desktop.
-

11. Modules

11.1 Citizen Module

Handles registration, login, complaint submission, tracking, and history.

11.2 Administrator Module

Handles admin login, complaint listing, filtering, status updates, and reporting.

11.3 Authentication Module

Manages role-based login (Citizen / Admin), sessions, and password hashing.

11.4 Complaint Management Module

Core module — handles complaint CRUD, image upload, category validation, and status transitions.

11.5 Dashboard Module

Renders summary counts, recent complaints, and charts for admins; personal history for citizens.

11.6 Database Module

Encapsulates MySQL connections, queries, and schema handling.

12. Workflow

```
Citizen
  ↓
  Login
  ↓
  Dashboard
  ↓
  Register Complaint
  ↓
  Complaint Stored
  ↓
  Backend Processing (Flask)
  ↓
  Database (MySQL)
  ↓
  Admin Dashboard
  ↓
  Status Update
  ↓
  Citizen Notification
  ↓
  Complaint Resolved
```

Step-by-step:

1. **Citizen** opens the web portal.
2. **Login** authenticates via Flask session.
3. **Dashboard** shows recent complaints and quick actions.
4. **Register Complaint** with category, description, location, image.
5. **Complaint Stored** — client-side validation, then submitted.
6. **Backend Processing** — Flask validates and persists to MySQL.
7. **Database** stores complaint with unique ID and timestamp.
8. **Admin Dashboard** reflects new complaints in real time.
9. **Status Update** — Admin marks as In Progress / Resolved.
10. **Citizen Notification** — updated status visible on citizen dashboard.
11. **Complaint Resolved** — logged in history.

13. Database Design

13.1 ER Diagram (Mermaid)

```
erDiagram
    USERS ||--o{ COMPLAINTS : "files"
    ADMINS ||--o{ STATUS_LOGS : "updates"
    COMPLAINTS ||--o{ STATUS_LOGS : "has"
    CATEGORIES ||--o{ COMPLAINTS : "classifies"

    USERS {
        int user_id PK
        varchar name
        varchar email
        varchar phone
        varchar password_hash
        datetime created_at
    }
    ADMINS {
        int admin_id PK
        varchar name
        varchar email
        varchar password_hash
    }
    CATEGORIES {
        int category_id PK
        varchar category_name
    }
    COMPLAINTS {
        int complaint_id PK
        int user_id FK
        int category_id FK
        text description
        varchar location
        varchar image_path
        enum status
        datetime created_at
        datetime updated_at
    }
    STATUS_LOGS {
        int log_id PK
        int complaint_id FK
```

```
int admin_id FK
enum old_status
enum new_status
datetime changed_at
}
```

13.2 Tables, Keys & Normalization

The design is in **3NF** — no repeating groups, non-key attributes fully depend on the primary key, and no transitive dependencies. Categories are separated to avoid duplication.

13.3 MySQL CREATE TABLE Statements

```
CREATE TABLE users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(120) UNIQUE NOT NULL,  
    phone VARCHAR(15),  
    password_hash VARCHAR(255) NOT NULL,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE admins (  
    admin_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(120) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE categories (  
    category_id INT AUTO_INCREMENT PRIMARY KEY,  
    category_name VARCHAR(60) UNIQUE NOT NULL  
);  
  
CREATE TABLE complaints (  
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    category_id INT NOT NULL,  
    description TEXT NOT NULL,  
    location VARCHAR(255) NOT NULL,  
    image_path VARCHAR(255),  
    status ENUM('Open', 'In Progress', 'Resolved') DEFAULT 'Open',  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (category_id) REFERENCES categories(category_id)  
);  
  
CREATE TABLE status_logs (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    complaint_id INT NOT NULL,  
    admin_id INT NOT NULL,  
    old_status ENUM('Open', 'In Progress', 'Resolved'),  
    new_status ENUM('Open', 'In Progress', 'Resolved'),
```

```
changed_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (complaint_id) REFERENCES complaints(complaint_id),  
FOREIGN KEY (admin_id) REFERENCES admins(admin_id)  
);
```

13.4 Sample Data

```
INSERT INTO categories (category_name) VALUES  
('Garbage Overflow'), ('Potholes'), ('Water Leakage'),  
('Street Light'), ('Drainage'), ('Illegal Parking'), ('Noise Pollution');  
  
INSERT INTO users (name,email,phone,password_hash) VALUES  
('Rahul Sharma','rahul@example.com','9876543210','<hashed>');  
  
INSERT INTO complaints (user_id,category_id,description,location,image_path,status)  
VALUES (1,2,'Large pothole near main gate','MG Road, Mumbai','uploads/1.jpg','Open');
```

14. UI Screens

Screen	Description
Login Page	Clean two-tab (Citizen / Admin) login with email + password. <i>[Screenshot Placeholder]</i>
Citizen Dashboard	Welcome card, quick "New Complaint" button, recent complaints list with color-coded status. <i>[Screenshot Placeholder]</i>
Complaint Form	Dropdown for category, description textarea, location field, image upload, submit button. <i>[Screenshot Placeholder]</i>
Complaint History	Table view: Complaint ID, Category, Date, Status, Action. <i>[Screenshot Placeholder]</i>
Admin Dashboard	Summary cards (Total / Open / In Progress / Resolved), searchable complaint table. <i>[Screenshot Placeholder]</i>
Complaint Details	Full view — image, description, location, timeline of status changes. <i>[Screenshot Placeholder]</i>
Status Update Page	Dropdown to change status with confirmation modal. <i>[Screenshot Placeholder]</i>

15. Algorithms

Note: No AI/ML model is used in this project. All logic is rule-based.

15.1 Complaint Processing Algorithm

1. Receive complaint form data.
2. Validate required fields (category, description, location, image).
3. Sanitize input (strip HTML, escape special chars).
4. Store image in /uploads/ with unique filename.
5. Insert record into complaints table with status='Open'.
6. Return unique complaint_id to citizen.

15.2 Status Update Logic (State Machine)

Allowed transitions:

Open → In Progress

In Progress → Resolved

(Backward transitions blocked)

On update:

1. Verify admin authentication.
2. Fetch current status.
3. Check transition legality.
4. Update complaints table.
5. Insert entry in status_logs.

15.3 Search Algorithm

1. Read search keyword & filters (category, status, date).
2. Build parameterized SQL query with LIKE and WHERE clauses.
3. Execute query with prepared statements (prevents SQL injection).
4. Return paginated results.

15.4 Validation Process

- **Client-side:** JS checks required fields, file type/size, email format.
- **Server-side:** Flask re-validates, hashes passwords, escapes strings.
- **Database-side:** ENUM, NOT NULL, and FK constraints as final guard.

16. Challenges Faced

Challenge	Solution
Time Constraints	Adopted agile sprints; split features across all 5 members.
Team Coordination	Daily 15-min stand-ups; used WhatsApp + GitHub Projects.
Backend Development	Started with Flask boilerplate and iterated feature-by-feature.
Frontend Integration	Defined REST endpoints early so frontend/backend could progress in parallel.
Database Design	Multiple ER diagram revisions before finalizing 3NF schema.
Testing	Manual test cases per module + Postman for APIs.
Debugging	Enabled Flask debug mode and used browser DevTools + MySQL Workbench.
GitHub Collaboration	Feature branches + pull requests to avoid merge conflicts.

17. Future Enhancements

- **Android Application** (Kotlin/Java or Flutter).
- **iOS Application** (Swift/Flutter).
- **GPS Integration** for auto-location capture.
- **Google Maps Integration** for pin-point complaint location.
- **Push Notifications** for status change alerts.
- **Cloud Deployment** on AWS / Azure / GCP.
- **Analytics Dashboard** with charts and hotspot maps.
- **AI Chatbot** for complaint guidance.
- **Multi-language Support** — Hindi, Marathi, English, and more.
- **Predictive Analytics** to forecast recurring issues.

18. Impact

18.1 Social Impact

Improves quality of life, empowers citizens, and builds trust in local governance.

18.2 Economic Impact

Reduces manpower cost of paperwork; faster resolution reduces long-term damage cost.

18.3 Environmental Impact

Paperless operation; timely garbage/drainage resolution reduces pollution & health hazards.

18.4 Business Impact

Reusable SaaS model — deployable to multiple municipalities.

18.5 Government Benefits

Data-driven decision making, better ward-level insights, alignment with Smart City Mission.

18.6 Citizen Benefits

Transparency, faster response, no queues, complete history at fingertips.

19. Results

19.1 System Performance

- Average complaint submission < 2 seconds.
- Admin dashboard loads < 3 seconds for 1,000+ records.
- Zero data loss with FK-enforced schema.

19.2 Testing Summary

Test Type	Cases	Pass
Unit Testing	25	25
Integration Testing	12	12
UI Testing	10	10
Security (basic)	6	6

19.3 Expected User Experience

Simple 3-step complaint filing, clear status indicators, minimal learning curve.

19.4 Demo Outcome

Live end-to-end demo: citizen files complaint → admin resolves → citizen sees updated status.

19.5 Project Benefits

Digital, transparent, scalable, and immediately deployable.

[Screenshot Placeholders: Login | Dashboard | Complaint Form | Admin Panel]

20. Conclusion

The **Smart Complaint Management System** successfully digitizes the civic complaint lifecycle, replacing an outdated manual process with a transparent, trackable, and citizen-friendly platform. It achieves every stated objective — reducing paperwork, improving transparency, engaging citizens, and enabling smarter governance. With planned enhancements such as mobile apps, GPS, and analytics, the system has strong potential to become a nationwide Smart City utility that positively impacts millions of citizens.

21. References

1. HTML Documentation — <https://developer.mozilla.org/en-US/docs/Web/HTML>
2. CSS Documentation — <https://developer.mozilla.org/en-US/docs/Web/CSS>
3. JavaScript Documentation — <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
4. Python Documentation — <https://docs.python.org/3/>
5. Flask Documentation — <https://flask.palletsprojects.com/>
6. MySQL Documentation — <https://dev.mysql.com/doc/>
7. Git Documentation — <https://git-scm.com/doc>
8. GitHub Documentation — <https://docs.github.com/>
9. Smart City Mission (India) — <https://smartcities.gov.in/>

22. Appendix

22.1 Sample API Responses

POST /api/complaints

```
{
  "status": "success",
  "complaint_id": 101,
  "message": "Complaint registered successfully"
}
```

GET /api/complaints/101

```
{  
  "complaint_id": 101,  
  "category": "Potholes",  
  "description": "Large pothole near main gate",  
  "location": "MG Road, Mumbai",  
  "image_url": "/uploads/101.jpg",  
  "status": "In Progress",  
  "created_at": "2026-02-11 10:24:15"  
}
```

22.2 Folder Structure

```
nexus-hackathon-2026/  
├─ app.py  
├─ requirements.txt  
├─ config.py  
├─ static/  
│   ├─ css/  
│   ├─ js/  
│   └─ uploads/  
├─ templates/  
│   ├─ login.html  
│   ├─ register.html  
│   ├─ citizen_dashboard.html  
│   ├─ complaint_form.html  
│   ├─ complaint_history.html  
│   └─ admin_dashboard.html  
├─ models/  
│   ├─ user.py  
│   ├─ complaint.py  
│   └─ admin.py  
├─ routes/  
│   ├─ auth_routes.py  
│   ├─ complaint_routes.py  
│   └─ admin_routes.py  
└─ database/  
    └─ schema.sql
```

22.3 Database Schema

See Section 13.3 for full MySQL DDL.

22.4 Code Snippets

Flask route — register complaint

```
@app.route('/api/complaints', methods=['POST'])
def create_complaint():
    if 'user_id' not in session:
        return jsonify({'error': 'Unauthorized'}), 401
    data = request.form
    image = request.files.get('image')
    filename = save_image(image)
    cursor.execute(
        "INSERT INTO complaints (user_id,category_id,description,location,image_path) "
        "VALUES (%s,%s,%s,%s,%s)",
        (session['user_id'], data['category_id'], data['description'],
        data['location'], filename))
    db.commit()
    return jsonify({'status':'success','complaint_id': cursor.lastrowid})
```

22.5 Project Workflow

See Section 12.

22.6 Glossary

Term	Meaning
SCMS	Smart Complaint Management System
ULB	Urban Local Body
CRUD	Create, Read, Update, Delete
3NF	Third Normal Form
SPV	Special Purpose Vehicle
FK / PK	Foreign Key / Primary Key

22.7 Abbreviations

Abbr.	Full Form
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
API	Application Programming Interface
DB	Database
UI	User Interface
UX	User Experience
DFD	Data Flow Diagram
ER	Entity Relationship
MVP	Minimum Viable Product